

Windows 逆向工程入门

实战篇

致即将踏入实战靶场的你：

在《基础篇》里，我们学会了如何“看懂”程序。你知道了变量在内存里如何排布，函数如何调用，汇编指令又是什么意思。那时候，你像是一个拿着解剖图的医学生，面对一具静止的“尸体”，可以头头是道地分析它的结构。

在《系统篇》里，我们学会了如何“理解”系统。你明白了虚拟内存的映射机制，PE 文件的加载过程，以及 API 调用的底层契约。那时候，你像是一个熟悉城市管网和交通规则规划师，能看懂这座数字城市是如何运转的。

但说实话，根据我对读者（或者说，对我自己）看书习惯的了解，你大概率没有把前两篇逐字读完、完全吃透，甚至可能是直接“空降”到了本篇章。

没关系。

现在我们要做的不再是分析和理解，而是“动手”。

对于实战篇，我绝不要求你必须“通关”前两篇才能开启。因为那太痛苦了。面对前面一大堆的“理论轰炸”，连作者本人都不可能坚持地看下去。

更别说，就算我写完了这套书，就能保证自己永远不忘吗？不可能的事。可能在未来的某一天，我也要返回来重读自己写的东西。

所以，我真正鼓励的是“边战边学”——你可以一边推进理论，一边在本书中进行验证。这两者不仅不冲突，反而是掌握逆向工程的最佳路径。

在这里，没有标准答案，没有温顺的程序。你将面对的是动态变化的内存、层层保护的进程、以及为了对抗你而存在的反作弊系统。

你将不再满足于“它是怎么运行的”，而是要亲手去验证“我能不能让它按我的想法运行”。

所以，忘掉那些按部就班的教程吧。真正的实战，从来都不是在一切都准备好之后才开始的。

正如生活一样。当我们面对一个目标时，总会想：“等我学会了所有知识，我再去做。”

可是我想说的是：有哪个事情能等到我们做好 100% 的准备时，才来到我们手里吗？

不可能的事。

但这阻挡不了我们在一件件没有准备好，甚至一点也不明白的事情中，带着害怕和紧张，勇敢地往前走。

勇敢地去想，去做，不要管他人的讥讽。就算他人不看好你，但这就是你整个人的全部吗？

不是吧，我的朋友。

本书的每一个小测试前，都会明确标注其对应的理论知识点。如果在实操中遇到盲区，你完全可以顺藤摸瓜，回过头去翻阅相关的理论章节。

随着前两篇内容的推进，本书会同步解锁对应的小测试。当这些测试全部展示完毕后，我们将迎来两个重磅章节：一是“自制 R3 调试器”，带你亲手拆解工具，看透调试器背后的底层工作原理；二是真正的综合实战，我将以经典游戏《植物大战僵尸》为靶场，为你完整复盘一次从逆向分析到破解的全流程。

回望计算机攻防安全的发展史，你会发现一个有趣的规律：攻的本质，是在不确定中寻找确定；而防的本质，是在确定中寻找不确定。

这不仅是技术的博弈，更是生活的隐喻。

所谓“确定感”，就像我们每天按部就班的生活轨迹，看似一成不变，但我们要学会在其中发现并享受那些微小的“不确定感”，那是平淡日子里的惊喜与波澜。

所谓“不确定感”，则像我们无法预知的明天。下一秒是平步青云，还是跌入低谷，我们无从得知。但正是这种未知，不妨碍我们在风雨飘摇中寻找内心的“确定感”，去建立属于自己的秩序与锚点。

在确定中拥抱未知，在未知中坚守确定。看似矛盾，但这，或许正是生活最迷人的本质吧。

带着这份对“确定与不确定”的理解，让我们正式进入《Windows 逆向工程入门实战篇》。

在接下来的内容中，我们将把理论化作具体的代码与操作。你将看到 C++ 对象与 x86 栈帧在内存中的真实布局，我们将亲手编写内存扫描器，并尝试通过管道、Socket、消息队列等多种方式，在进程之间建立隐秘的通信。此外，我们还会探讨各种注入技术与 Hook 机制，去观察程序逻辑被篡改时的底层反应。

C++ 逆向实战

1. 类和对象 变量成员内存布局穿透
2. 类和对象 static 成员内存布局
3. 类和对象 new delete 实现方法
4. 类和对象 虚拟表函数劫持

5. x86 函数栈帧 视图
6. x86 函数栈帧 rop + shellcode 劫持
7. 内存扫描器
8. PE 文件格式解析
9. 导入表攻防
10. thread APC 劫持
11. thread 上下文劫持
12. 指针链读取 —— Read/WriteProcessMemory
13. 指针链写入 —— Read/WriteProcessMemory
14. 指针链读取 —— Socket
15. 指针链写入 —— Socket
16. 指针链读取 —— Pipe
17. 指针链写入 —— Pipe
18. 指针链读取 —— Message
19. 指针链写入 —— Message
20. 特征码读取 —— Read/WriteProcessMemory
21. 特征码写入 —— Read/WriteProcessMemory
22. 特征码读取 —— Socket
23. 特征码写入 —— Socket
24. 特征码读取 —— Pipe
25. 特征码写入 —— Pipe
26. 特征码读取 —— Message
27. 特征码写入 —— Message
28. 篡改程序逻辑 —— Memory
29. 篡改程序逻辑 —— File
30. 普通注入 —— LoadLibrary
31. 汇编码注入 —— LoadLibrary
32. APC 注入 —— LoadLibrary
33. 线程劫持注入 —— LoadLibrary
34. 普通注入 —— Manual mapping
35. 汇编码注入 —— Manual mapping
36. APC 注入 —— Manual mapping
37. 线程劫持注入 —— Manual mapping
38. Windows 消息 Hook
39. PE 导入表 IAT Hook
40. inline Hook

朋友们，期待已久的逆向实战小项目终于要开始了！接下来的旅程我们将全面转向‘真刀真枪’的代码实操。为了保持实战的连贯性，相关的理论铺垫会逐步省略，让我们直接上手，在实战中检验真知！

C++逆向实战

1. 类和对象：变量成员内存布局穿透

在进行实战前，建议先掌握以下理论基础：

- 全部章节
 - [基础篇] 第一章：C++——掌控内存的“降维”语言
 - [基础篇] 第二章：数据类型的本质——内存的计量单位
 - [基础篇] 第三章：数组与指针——连续空间与移动标尺
 - [基础篇] 第四章：类与对象——内存里的“伪装大师”
- 重点章节
 - [基础篇] 2.2 内存对齐：为了速度的“空间浪费”
 - [基础篇] 3.5 指针与数组的“易容术”（指针数组）
 - [基础篇] 4.1 类的真相：不存在的“类”，真实的“结构体”
 - [基础篇] 4.2 访问控制：纸老虎般的 `public`, `private`, `protected`

我们的目标就直接锁定在对象的成员变量上。通过精心构造的指针，我们可以绕过 C++ 的访问控制，直接读写对象的私有数据。

这在逆向工程和游戏外挂中是非常实用的基础技术。

核心思路：指针偏移与内存布局

C++ 的 `private` 关键字只在编译期起作用，是一种语法层面的访问限制。在程序运行时，对象在内存中就是一块连续的区域，所有的成员变量（无论 `public` 还是 `private`）都按顺序排列在其中。

攻击的本质：只要我们知道了对对象的起始地址(this 指针)，并计算出目标私有变量在对象内的内存偏移量，就可以通过指针运算直接定位并修改它。

实战代码

```
/**
 * @file ClassAndObjectTest-Access.cpp
 * @brief 演示 C++ 内存布局穿透 (Memory Layout Penetration) 与访问控制 bypass
 *
 * @details
 * 本文件通过指针算术 (Pointer Arithmetic) 和类型强制转换 (Type Casting),
 * 直接操作对象在内存中的物理地址, 从而绕过编译期的访问控制检查 (private/protected)。
 *
 * 核心原理:
 * 1. C++ 的对象内存布局是连续且确定的 (Standard Layout)。
```

```

* 2. 访问修饰符 (public/private/protected) 仅为编译期语法约束，不占用运行时内存。
* 3. 只要计算出成员变量相对于对象首地址的偏移量 (Offset), 即可通过裸指针直接读写。
*
* 技术关键词:
* - Memory Layout Penetration (内存布局穿透)
* - Offset-based Access (基于偏移量的访问)
* - Type Punning (类型双关)
* - Encapsulation Bypass (封装绕过)
*
* @warning
* 此操作破坏了封装性，依赖于特定的编译器内存对齐策略。
* 若类定义变更或编译器优化策略改变，偏移量计算将失效，导致未定义行为 (UB)。
* 仅用于底层原理研究、调试或逆向工程，严禁在生产环境业务逻辑中使用。
*
* Let's Debug!
*   -> 操作路径: Visual Studio [调试] -> [窗口] -> [内存/即时/自动]
*   -> 目标: 亲眼见证指针偏移如何绕过 private, protected 限制修改内存!
*
* @author cnHHHHHcn
* @date 2026-02-27
*/
#include <iostream>

class Target {
// this 是常量指针，不能更改 as
public:
    int a = 0xAAAAAAAA;
    void Func() {
        Init();
        std::cout << "Debug Func" << std::endl;
        std::cout << "Public a:" << a << std::endl;
        std::cout << "Private b:" << b << std::endl;
        std::cout << "Protected c:" << c << std::endl;
    }
private:
    short b = 0xBBBB;
    void Init() {
        std::cout << "Class Target Object Address:" << this << std::endl;
    }
protected:
    char c = 0x43;
};

```

```

int main()
{
    std::cout << "[ Reverse Engineering Project : Memory Layout Penetration ]\n" <<
    std::endl;

    Target tg1, tg2;
    char* ptg = nullptr;
    std::cout << std::hex;

    ptg = (char*)&tg1;
    std::cout << "tg1 Address:" << (void*)ptg << std::endl;
    std::cout << "=== [Object tg1 before Penetration] ===" << std::endl;
    tg1.Func();

    // --- 开始内存穿透操作 ---
    tg1.a = 0x11111111;
    // 偏移量 sizeof(int) 即跳过 public 成员 'a', 直接写入 private 成员 'b' (short)
    *(short*)(ptg + sizeof(int)) = 0xB00B;
    // 偏移量 sizeof(int) + sizeof(short) 跳过 'a' 和 'b', 直接写入 protected 成员 'c'
    (char)
    *(char*)(ptg + sizeof(int) + sizeof(short)) = 0x41;
    // --- 内存穿透结束 ---

    std::cout << "=== [Object tg1 After Penetration] ===" << std::endl;
    tg1.Func();

    std::cout << std::endl;

    ptg = (char*)&tg2;
    std::cout << "tg2 Address:" << (void*)ptg << std::endl;
    std::cout << "=== [Object tg2 before Penetration] ===" << std::endl;
    tg2.Func();

    // --- 开始内存穿透操作 ---
    tg2.a = 0x22222222;
    // 同样逻辑: 跳过 int a, 修改 private short b
    *(short*)(ptg + sizeof(int)) = 0xBB00;
    // 同样逻辑: 跳过 int a + short b, 修改 protected char c
    *(char*)(ptg + sizeof(int) + sizeof(short)) = 0x42;
    // --- 内存穿透结束 ---

    std::cout << "=== [Object tg2 After Penetration] ===" << std::endl;
    tg2.Func();
    return 0;
}

```

```
}
```

核心关键点

- * 偏移量的准确性：这是成败的关键。你必须精确分析出变量在对象中的偏移。这通常需要通过分析构造函数、成员函数对变量的访问方式（如 `mov eax, [ecx+1Ch]`）来确定。
- * 内存对齐 (Padding)：不要假设变量是紧密排列的。编译器为了性能会对变量进行内存对齐，中间可能会插入填充字节。逆向时必须考虑到这一点，否则你会改到错误的变量上。
- * `thiscall` 调用约定：在逆向成员函数时，你经常会看到 `ecx` 寄存器 (x86) 被用来传递 `this` 指针。找到 `this`，就找到了对象的基址。

2. 类和对象 new delete 实现方法

在进行实战前，建议先掌握以下理论基础：

- 全部章节
 - [基础篇] 第一章：C++——掌控内存的“降维”语言
 - [基础篇] 第二章：数据类型的本质——内存的计量单位
 - [基础篇] 第三章：数组与指针——连续空间与移动标尺
 - [基础篇] 第四章：类与对象——内存里的“伪装大师”
- 重点章节
 - [基础篇] 2.2 内存对齐：为了速度的“空间浪费”
 - [基础篇] 3.5 指针与数组的“易容术”（指针数组）
 - [基础篇] 4.1 类的真相：不存在的“类”，真实的“结构体”
 - [基础篇] 4.2 访问控制：纸老虎般的 `public`, `private`, `protected`

实战代码

```
/**
 * @file ClassAndObjectTest-StaticMember.cpp
 * @brief C++ 静态成员变量底层原理验证实验
 *
 * @details
 * 本文件旨在通过内存地址分析，深入验证 C++ 静态成员变量 (static member variables)
 * 的存储机制、生命周期及与对象实例的独立性。
 *
 * 核心验证点：
 * 1. 【内存独立性】验证静态成员存储于全局数据区 (.data/.bss)，其地址完全独立于栈/堆
 * 上的对象实例。
 * 2. 【共享性验证】证明所有对象实例 (tg1, tg2) 访问静态成员时，指向的是同一物理内
 * 存地址。
 * 3. 【空间不占用】通过 sizeof 运算，证实静态成员不计入对象实例的内存大小。
 * 4. 【地址范围计算】手动计算并打印静态成员在全局区的连续内存分布范围。
 * 5. 【类外定义机制】演示静态成员必须在类外进行定义和初始化的链接规则。
 *
 * 预期现象：
 * - &tg1.s_a == &tg2.s_a == &Target::s_a
```

```

* - sizeof(Target) 不包含 static 成员的大小
* - 修改任意对象的 static 成员，所有对象及类名访问的值同步改变
*
* @warning
* 本实验涉及底层内存地址打印与分析，输出结果依赖于具体编译器实现及内存对齐策略。
* 静态成员在多线程环境下非线程安全 (Race Condition)，本测试仅在单线程下运行。
*
* Let's Debug!
*   -> 操作路径: Visual Studio [调试] -> [窗口] -> [内存/即时/自动]
*   -> 目标: 亲眼见证静态成员在全局区的独立地址!
*
* @author cnHHHHHcn
* @date 2026-02-27
* @version 1.0
*/

#include <iostream>

class Target {
/**
* @brief 核心区别总结: 静态 vs 普通 成员函数
*
* 1. 根本差异: 静态函数无 `this` 指针，故不依赖对象实例。
* 2. 访问权限: 静态函数仅能访问静态成员（无法访问非静态成员）。
* 3. 调用方式: 静态函数可通过类名直接调用，无需构造对象。
* 4. 多态限制: 静态函数不能是虚函数 (virtual)，不支持重写与动态绑定。
*
* 总结: 静态函数属于“类”，普通函数属于“对象”。
*/
public:
    int a = 0xAAAAAAAA;
    static char s_a;
    static short s_b;
    static int s_c;
    // static char test = 'T'; // Error: 带有类内初始值设定项的成员必须为常量
private:
    short b = 0xB BBB;
protected:
    char c = 0x43;
};

// =====

```



```

// 【关键步骤】类外定义与初始化
// 格式：类型 类名::成员名 = 初始值;
// 这一步才真正分配了内存，解决了 "无法解析的外部符号" 错误
// 定义并初始化 public 静态成员
// =====
char Target::s_a = 0x41;
short Target::s_b = 0x4242;
int Target::s_c = 0x43434343;

int main() {
    Target tg1, tg2;

    std::cout << "[ Reverse Engineering Project : Static Variable Research ]\n" << std::endl;

    std::cout << std::hex;
    // void* pTargetStaticVariable = &tg1.s_a;
    // void* ptg = &tg1;
    // void* ptg = &tg2;

    // [验证点 1 & 2] 打印地址，验证所有实例与类名访问的都是同一个全局地址
    std::cout << "Class Target Public Static s_a Address: " << (void*)&Target::s_a << std::endl;
    std::cout << "Instance Object tg1 Public Static s_a Address: " << (void*)&tg1.s_a <<
std::endl;
    std::cout << "Instance Object tg2 Public Static s_a Address: " << (void*)&tg2.s_a <<
std::endl;
    std::cout << std::endl;

    // [验证点 2] 打印初始值，确认数据一致
    std::cout << "Class Target Public Static s_a: " << Target::s_a << std::endl;
    std::cout << "Instance Object tg1 Public Static s_a: " << (char)tg1.s_a << std::endl;
    std::cout << "Instance Object tg2 Public Static s_a: " << (char)tg2.s_a << std::endl;

    // [验证点 2] 修改其中一个实例的静态成员，观察全局变化
    tg1.s_a = 0x30;
    std::cout << std::endl;
    std::cout << "Change Instance Object tg1 Public s_a" << std::endl;
    std::cout << "Class Target Public Static s_a: " << Target::s_a << std::endl;
    std::cout << "Instance Object tg1 Public Static s_a: " << (char)tg1.s_a << std::endl;
    std::cout << "Instance Object tg2 Public Static s_a: " << (char)tg2.s_a << std::endl;

    std::cout << std::endl;

    // [验证点 3] 计算类大小，证明 static 成员不占用对象实例空间
    size_t ClassVariableSize = sizeof(Target);

```

```

    std::cout << "Class Target Variable Size: " << (size_t)ClassVariableSize << " Bytes" <<
    std::endl;

    // 打印实例在栈上的内存范围
    std::cout << "Instance Object tg1 Variable Memory Range: " << (void*)&tg1 << " - " <<
    (void*)((char*)&tg1 + ClassVariableSize) << std::endl;
    std::cout << "Instance Object tg2 Variable Memory Range: " << (void*)&tg2 << " - " <<
    (void*)((char*)&tg2 + ClassVariableSize) << std::endl;

    std::cout << std::endl;

    // [验证点 4] 打印静态成员在全局区的地址分布
    std::cout << "Class Target Public Static s_a Address: " << (void*)&Target::s_a << " (Size:
1 Bytes; Type: char)" << std::endl;
    std::cout << "Class Target Public Static s_b Address: " << (void*)&Target::s_b << " (Size:
2 Bytes; Type: short)" << std::endl;
    std::cout << "Class Target Public Static s_c Address: " << (void*)&Target::s_c << " (Size:
4 Bytes; Type: int)" << std::endl;

    // 计算静态成员占用的总内存范围
    size_t ClassStaticVariableSize = ((char*)&Target::s_c - (char*)&Target::s_a) + sizeof(int);
    std::cout << "Class Target Static Variable Size: " << std::dec << ClassStaticVariableSize
<< " Bytes" << std::hex << std::endl;
    std::cout << "Class Target Static Variable Memory Range: " << (void*)&Target::s_a << "
- " << (void*)((char*)&Target::s_a + ClassStaticVariableSize) << std::endl;

    return 0;
}

```

3. 类和对象 new delete 实现方法

在进行实战前，建议先掌握以下理论基础：

- 全部章节
 - [基础篇] 第一章：C++——掌控内存的“降维”语言
 - [基础篇] 第二章：数据类型的本质——内存的计量单位
 - [基础篇] 第三章：数组与指针——连续空间与移动标尺
 - [基础篇] 第四章：类与对象——内存里的“伪装大师”
- 重点章节
 - [基础篇] 2.2 内存对齐：为了速度的“空间浪费”
 - [基础篇] 3.5 指针与数组的“易容术”（指针数组）
 - [基础篇] 4.1 类的真相：不存在的“类”，真实的“结构体”
 - [基础篇] 4.2 访问控制：纸老虎般的 public, private, protected

在逆向工程的视角下，C++ 的 new 和 delete 远不止是申请和释放内存那么简单。它们是

“分配内存 + 调用构造/析构函数”的复合体。

为了在逆向时准确识别并还原程序逻辑，你需要掌握以下三个核心逆向思路：

思路一：拆解“复合动作” (Placement New 的逆向)

new 和 delete 实际上是由内存分配/释放和构造/析构两个独立步骤组成的。在逆向时，你要学会将它们“拆”开来看。

* new 的底层流程：

1. 分配原始内存：调用 operator new (通常底层封装了 malloc)。
2. 构造对象：在分配好的内存地址上，调用对象的构造函数。

* 逆向特征：你可能会看到先调用 malloc (或自定义分配器)，紧接着就是一大段复杂的初始化代码（构造函数逻辑）。

* delete 的底层流程：

1. 析构对象：调用对象的析构函数（清理资源，如释放成员指针）。
2. 释放内存：调用 operator delete (通常底层封装了 free)。

* 逆向特征：先看到一段清理逻辑（析构函数），再看到 free 调用。

逆向技巧：

如果遇到 free 之后紧接着有复杂的逻辑，或者 malloc 之前有复杂的清理代码，那很可能你看到的是编译器内联展开的 delete 或 new 逻辑，而不是单纯的 C 语言 malloc/free。

思路二：识别“数组的元信息” (New[] 的坑)

这是逆向 C++ 程序最容易翻车的地方。new[] 和 delete[] 的实现包含了一个隐藏的元信息 (Metadata) 机制。

* new T[N] 的内存布局：

如果 T 是一个需要析构的类 (有析构函数)，编译器分配的内存通常比 $N * \text{sizeof}(T)$ 更大。

* 头部存储：在返回的数组指针前面（通常是 -4 或 -8 字节处），编译器会偷偷写入数组长度 N。

* 目的：为了让 delete[] 知道需要调用多少次析构函数。

* 逆向识别点：

* 分配大小异常：如果你看到分配的内存大小是 0x28 (40) 字节，但对象大小是 0x24 (36) 字节，多出来的 4 字节很可能就是用来存数组长度的。

* 指针偏移：在汇编代码中，你可能会看到类似 mov [eax-4], 5 的指令（将 5 写入指针前 4 字节），这通常就是在存储数组长度。

* delete[] 的逻辑：delete[] 会先读取这块隐藏的长度信息，然后倒序循环调用析构函数，最后调用 operator delete 释放整个块（包括前面存长度的那几个字节）。

经典错误匹配：

如果用 delete 释放 new[] 分配的数组：

- * 后果：delete 不会读取数组长度，只会调用一次析构函数，然后直接 free。
- * 逆向表现：这会导致内存泄漏（只析构了第一个对象）或堆损坏（如果 free 的指针不是块首地址，但在现代编译器中，由于 new[] 返回的是数据区指针，直接 delete 可能会导致 free 传参错误或只释放部分内存）。

思路三：追踪“运算符重载”（自定义内存池）

在大型游戏或软件中，全局的 new/delete 经常被重载，或者类内部有自己的内存管理。这是逆向分析内存管理机制的重点。

* 全局重载：

* 现象：你在代码中找不到对 malloc 或 HeapAlloc 的直接调用，而是调用了某个特定的函数（如 MyAlloc、GMEM_Alloc）。

* 分析：这就是重载了全局 operator new。你需要去分析这个自定义分配器的逻辑，它可能涉及内存池、对齐处理或日志记录。

* 类专属重载：

* 现象：某个特定类的 new 操作行为与其他类不同。

* 代码特征：

```
class Monster {
public:
    void* operator new(size_t size) {
        // 使用怪物专属的内存池
        return MonsterPool.Alloc(size);
    }
};
```

* 逆向策略：当你发现某个类的创建/销毁逻辑异常复杂，或者分配的内存地址总是落在特定区间时，检查该类是否实现了自己的 operator new/delete。

总结：逆向 new/delete 的检查清单

1. 看类型：是内置类型（int, char）还是自定义类？

* 内置类型：new $\approx$ malloc, delete $\approx$ free。

* 自定义类：必定伴随构造/析构函数调用。

2. 看数组：是否是 new[]？

* 检查分配的内存大小是否比对象大小多出 4/8 字节。

* 检查指针附近是否有存储长度的整数。

3. 看分配器：底层调用的是 malloc 还是其他函数？

* 如果是其他函数，说明程序使用了自定义内存管理（内存池），需要重点分析该函数的逻辑。

实战代码

```
/**
 * @file ClassAndObject-ManualMemory.cpp
 * @brief C++ 类对象内存布局解析与手动生命周期管理实验
 *
 * @details
 * 本文件演示了在不依赖编译器自动生成的构造/析构函数的情况下,
 * 如何通过 C 风格的手动内存操作来模拟 C++ 类的完整生命周期。
 * 核心机制包括:
 * 1. 【原始内存分配】使用 calloc 在堆上分配未初始化的原始内存块。
 * 2. 【手动构造模拟】通过全局函数 TargetInitiate 强行写入内存, 模拟构造函数行为。
 * 3. 【内存布局穿透】利用指针偏移 (Pointer Arithmetic) 直接访问 private/protected 成员。
 * 4. 【手动析构模拟】通过全局函数 TargetDestory 清理数据并 free 内存。
 *
 * 【重要说明 / CRITICAL NOTE】
 * 1. **非 RAII 实现**:  

 *   - 真正的 RAII (Resource Acquisition Is Initialization) 应利用栈对象析构自动释放资源。  

 *   - 本代码依赖程序员显式调用 ClassDeleteOpera, 若忘记调用将导致内存泄漏。  

 *   - 此模式常见于逆向工程分析 (还原编译器底层行为) 或嵌入式受限环境, 而非现代 C++ 最佳实践。
 * 2. **硬编码偏移量风险**:  

 *   - 代码中 *(short*)((char*)pObject + 0x4) 强依赖特定编译器 (MSVC) 的内存对齐策略。  

 *   - 若成员变量类型或顺序改变, 偏移量 0x4/0x6 将失效, 导致数据错乱。
 * 3. **学习价值**:  

 *   - 帮助理解 this 指针的本质 (即对象首地址)。  

 *   - 揭示 C++ 成员变量在内存中的真实排列顺序 (Public/Private/Protected 无区别, 仅访问权限不同)。
 *
 * @warning
 * 本代码涉及底层内存读写, 包含硬编码偏移量和手动内存管理。
 * 仅用于逆向原理研究与教学, 严禁在生产环境使用此类手动管理方式。
 *
 * Let's Debug!
 * -> 操作路径: Visual Studio [调试] -> [窗口] -> [内存 1] (Memory 1)
 * -> 目标:
 * 1. 【见证内存布局】: 在 TargetInitiate 执行后, 观察内存窗口。  

 *   确认 0x00 处是 int a (4 字节), 0x04 处是 short b, 0x06 处是 char c。  

 *   验证访问控制符 (public/private) 不影响内存物理位置。
 * 2. 【追踪指针变换】: 单步执行 ClassNewOpera。  

 *   观察 calloc 返回的 void* 如何被强制转换为 Target* 并传入初始化函数。
 * 3. 【监控生命周期】: 在 main 函数末尾, 确认 ClassDeleteOpera 是否将内存清零并释放。
```

```

*           对比若不调用该函数，内存窗口中的数据是否残留（内存泄漏）。
*
* @author cnHHHHHcn
* @date 2026-02-27
* @version 1.0 (Experimental - Manual Memory Management)
*/

#include <iostream>
#include <stdlib.h>

// ===== 目标类定义（内存布局模板） =====
/**
 * @brief 用于测试内存布局的靶子类
 * @note 此类故意不写构造函数/析构函数，以便演示手动初始化过程
 */
class Target {
public:
    int a;           // 偏移量 0x00 (4 字节)

    // 普通成员函数：第一个参数隐含 this 指针
    void OutThisPointer() {
        std::cout << "Current instance Address (this): " << this << std::endl;
    };

    void OutMemberInfo() {
        std::cout << "Public int a (Offset 0x0): " << a << std::endl;
        // 以下两行虽然能访问私有/保护成员，但演示了通过 this 指针的直接访问
        std::cout << "Private short b (Offset 0x4): " << b << std::endl;
        std::cout << "Protected char c (Offset 0x6): " << c << std::endl;
    };

private:
    short b;         // 偏移量 0x04 (2 字节) - 注意：受对齐影响，可能占用 2 字节

protected:
    char c;          // 偏移量 0x06 (1 字节) - 注意：后续可能有填充字节(padding)
};

// ===== 手动构造与析构模拟 =====

/**
 * @brief 模拟构造函数：手动初始化内存布局
 * @param pObject 指向已分配但未初始化内存的指针
 * @warning 硬编码偏移量 (0x4, 0x6) 依赖 MSVC x64 默认对齐策略

```

```

*/
void TargetInitiate(Target* pObject) {
    if (!pObject) return;

    // 1. 初始化 public 成员 (直接访问)
    pObject->a = 0x11111111;

    // 2. 初始化 private 成员 (通过指针偏移暴力写入)
    // 逻辑: (char*)pObject 转为字节指针 -> +4 跳过 int a -> 强转为 short* -> 赋值
    *(short*)((char*)pObject + 0x4) = 0x2222;

    // 3. 初始化 protected 成员 (通过指针偏移暴力写入)
    // 逻辑: +6 跳过 int a(4) + short b(2) -> 强转为 char* (实际代码里强转的是 short*取
    // 低 8 位, 这里修正为 char*)
    // 原代码逻辑: *(short*)((char*)pObject + 0x6) = 0x33;
    // 修正说明: 原代码用 short* 接收 0x33 没问题, 但语义上 c 是 char。
    // 保持原逻辑以匹配你的测试值, 但需注意这里实际上写入了 2 字节 (0x33 0x00)
    *(unsigned char*)((char*)pObject + 0x6) = 0x33;
};

/**
 * @brief 模拟析构函数: 手动清理内存数据
 * @param pObject 指向待释放对象的指针
 */
void TargetDestory(Target* pObject) {
    if (!pObject) return;

    // 清零数据, 防止释放后残留敏感信息 (虽然 free 后不应再访问)
    pObject->a = 0;
    *(short*)((char*)pObject + 0x4) = 0;
    *(unsigned char*)((char*)pObject + 0x6) = 0;
};

// ===== 内存管理接口 (C-Style API) =====

/**
 * @brief 封装内存分配与初始化 (类似 new 操作符的重载)
 * @return 指向已初始化对象的指针
 */
Target* ClassNewOpera() {
    // 1. 分配原始内存 (内容为 0)
    void* AllocAddress = calloc(1, sizeof(Target));
    if (!AllocAddress) return nullptr;

```

```

        // 2. 手动调用“构造函数”
        TargetInitiate((Target*)AllocAddress);

        return (Target*)AllocAddress;
    }

    /**
     * @brief 封装清理与释放 (类似 delete 操作符的重载)
     * @param pObject 待释放的对象指针
     */
    void ClassDeleteOpera(Target* pObject) {
        if (!pObject) return;

        // 1. 手动调用“析构函数”清理数据
        TargetDestory(pObject);

        // 2. 释放堆内存
        free((void*)pObject);
    }

    // ===== 主测试入口 =====
    int main(){

        std::cout << "[ Reverse Engineering Project : ManualMemory ]" << std::endl;

        // 1. 手动创建对象 (模拟 new Target())
        Target* tg = ClassNewOpera();    // 后续输出使用十六进制, 方便观察内存地址
        std::cout << std::hex;

        // 2. 验证对象信息
        std::cout << "Class Target Initiated Instance Object tg" << std::endl;
        std::cout << std::endl;
        std::cout << "Instance Object tg Address: " << tg << std::endl;

        // 验证 this 指针是否等于对象地址
        tg->OutThisPointer();
        std::cout << std::endl;
        std::cout << "Instance Object tg Variable Overall" << std::endl;

        // 验证能否正确读取通过偏移量写入的数据
        tg->OutMemberInfo();
        std::cout << std::endl;

        // 3. 手动销毁对象 (模拟 delete tg)

```



```
// 【关键】：如果不执行这一行，将发生内存泄漏 (Memory Leak)
ClassDeleteOpera(tg);
std::cout << "Released Instance Object tg" << std::endl;
return 0;
}
```

3. 类和对象 虚拟表函数篡改

在进行实战前，建议先掌握以下理论基础：

- 全部章节
 - [基础篇] 第一章：C++——掌控内存的“降维”语言
 - [基础篇] 第二章：数据类型的本质——内存的计量单位
 - [基础篇] 第三章：数组与指针——连续空间与移动标尺
 - [基础篇] 第四章：类与对象——内存里的“伪装大师”
- 重点章节
 - [基础篇] 2.2 内存对齐：为了速度的“空间浪费”
 - [基础篇] 3.5 指针与数组的“易容术”（指针数组）
 - [基础篇] 4.1 类的真相：不存在的“类”，真实的“结构体”
 - [基础篇] 4.2 访问控制：纸老虎般的 public, private, protected
 - [基础篇] 4.3 this 指针：对象的“身份证”
 - [基础篇] 4.4 虚函数与虚表 (vtable)：多态的“后门”

核心原理：C++多态的“命门”

在 C++ 中，多态是通过虚函数表 (VTable) 和虚指针 (VPtr) 实现的：

VTable (虚表)：编译器为每一个包含虚函数的类生成一个全局的函数指针数组。数组里存的是该类所有虚函数的真实地址。

VPtr (虚指针)：该类的每一个对象在内存中（通常是头部）都会包含一个隐藏指针，指向其所属类的 VTable 。

攻击的本质：既然程序通过 对象 -> VPtr -> VTable -> 函数地址 这条链路来调用虚函数，那么只要我们能修改链路中的任意一环（通常是修改对象的 VPtr，或者修改 VTable 里的函数指针），就能让程序去执行我们指定的代码。

```
/**
 * @file ClassAndObjectTest-VirtualHook.cpp
 * @brief C++ 虚函数表劫持 (VTable Hooking) 与内存布局穿透实验
 *
 * @details
 * 本文件演示了如何通过直接操作对象内存布局，实现运行时多态的“劫持”。
 * 核心机制包括：
 * 1. 【虚表解析】遍历对象头部的 vptr，提取虚函数地址列表。
 * 2. 【内存伪造】在堆 (Heap) 上动态分配可写内存，复制原始虚表内容。
```

```

* 3. 【指针重定向】修改对象 vptr，使其指向伪造的虚表，从而拦截函数调用。
* 4. 【偏移量访问】通过硬编码偏移量 (0x8) 直接读写 protected 成员变量 (m_Value)。
*
* 【高危警告 / CRITICAL WARNING】
* 1. **非生产代码**：本代码仅用于底层原理研究、逆向工程学习或调试测试。
*   严禁在任何生产环境、商业项目或稳定系统中使用此类技术。
* 2. **未定义行为 (UB)**：
*   - 硬编码偏移量 (0x8) 依赖于特定的编译器版本、架构 (x64) 及对齐策略。
*   - 更换环境极易导致内存越界访问 (Access Violation)。
* 3. **内存管理风险**：
*   - 手动 calloc/free 虚表内存极易引发内存泄漏或双重释放 (Double Free)。
*   - 若对象析构时未恢复原始 vptr，将导致程序崩溃。
* 4. **安全性**：此类技术常被恶意软件利用，编译后可能被杀毒软件误报为病毒。
*
* @warning
* 本代码涉及底层内存操作，包含硬编码偏移量和手动内存管理。
* 仅用于技术与教学，严禁在生产环境使用。
*
* Let's Debug!
*   -> 操作路径: Visual Studio [调试] -> [窗口] -> [内存 1] (Memory 1) & [即时]
(Immediate)
*   -> 目标：
*       1. 【见证偷梁换柱】：在 CheatClassVirtualFunc 执行前后，对比对象头部 (vptr)
的值。
*       观察它如何从 .rdata (只读代码段) 跳变到 Heap (堆内存)，证明虚表已被伪造。
*       2. 【验证函数拦截】：在 Dtg->Add() 处下断点，单步执行 (F10/F11)。
*       确认指令跳转地址不再是 DerivedTarget::Add，而是我们的 Multiply 函数。
*       3. 【透视内存穿透】：在 Multiply/Divide 函数内，查看 (char*)Dtg + 0x8 处的内
存值。
*       亲眼见证 double 类型的 m_Value 如何被直接暴力改写，绕过所有封装保护。
*       4. 【监控生命周期】：观察 ReleaseHeap 逻辑执行时，旧堆内存是否被正确释放，
*       警惕 Access Violation (0xC0000005) 在错误释放只读内存时爆发。
* - 代码中的 `ReleaseHeap` 参数仅为演示逻辑，实际应用中需更严谨的生命周期管理。
*
* @author cnHHHHHcn
* @date 2026-02-27
* @version 1.0 (Experimental)
*/

#include <iostream>
#include <vector>
#include <windows.h>

// ===== 基类定义 =====

```

```

class BaseTarget {
public:
    // 虚函数：会在虚表中占据位置 (索引 0)
    virtual void Add(double Param) {};
    // 虚函数：会在虚表中占据位置 (索引 1)
    virtual void Sub(double Param) {};

    // 普通成员函数：不在虚表中，直接调用
    double GetValue() {
        return m_Value;
    };

protected:
    // 受保护成员：派生类可访问，位于对象内存中
    double m_Value = 0;
};

// ===== 派生类定义 =====
class DerivedTarget : public BaseTarget {
public:
    // 重写基类虚函数
    void Add(double Param) override {
        m_Value += Param;
    };
    void Sub(double Param) override {
        m_Value -= Param;
    };
};

// 类型别名：虚函数表列表 (存储函数地址)
typedef std::vector<void*> VTFList;

// ===== 黑客函数：直接内存修改 =====
/**
 * @brief 模拟乘法操作 (通过指针偏移直接修改内存)
 * @warning 硬编码偏移量 0x8 依赖特定编译器/架构 (64 位下 vptr=8 字节, 无 padding)
 */
void Multiply(DerivedTarget* Dtg, double Param) {
    // 1. 计算 m_Value 的地址：对象首地址 + 8 字节 (跳过 vptr)
    // 注意：这是不安全的硬编码，推荐使用 offsetof
    void* ptg_m_Value = ((char*)Dtg + 0x8);

    // 2. 强制类型转换为 double 指针
    double* tg_m_Value = (double*)ptg_m_Value;

```

```

// 3. 直接修改内存中的值
*tg_m_Value *= Param;
}

/**
 * @brief 模拟除法操作 (通过指针偏移直接修改内存)
 */
void Divide(DerivedTarget* Dtg, double Param) {
    void* ptg_m_Value = ((char*)Dtg + 0x8);
    double* tg_m_Value = (double*)ptg_m_Value;
    *tg_m_Value /= Param;
}

// ===== 虚表解析工具 =====
/**
 * @brief 遍历虚函数表, 提取所有函数地址
 * @warning MSVC 的虚表通常不以 nullptr 结尾, 此处的 while(FnAddress) 可能导致越界
        读取
        *           仅在虚表末尾恰好为 0 或遇到非法地址崩溃前能工作 (极度危险)
        */
VTFList GetClassVirtualTable(void* pVirtualTableAddr) {
    void* FnAddress = nullptr;
    VTFList VirtualTableFuncList;
    int maxCount = 20; // 假设虚函数不会超过 20 个, 防止死循环
    do {
        // 取出当前索引的函数地址
        FnAddress = *(void**)pVirtualTableAddr;

        // 如果地址有效则加入列表 (若为 0 则停止)
        VirtualTableFuncList.push_back(FnAddress);

        // 指针后移一个单位 (sizeof(void*)), 指向下一个函数指针
        pVirtualTableAddr = (void*)((unsigned long long)pVirtualTableAddr + sizeof(char*));
        if (VirtualTableFuncList.size() > maxCount) break; // 安全阀
    } while (FnAddress); // 危险: 依赖空指针作为结束标志

    return VirtualTableFuncList;
}

/**
 * @brief 打印对象内存布局及虚表详情
 */
void DisplayClassVirtualFunc(void* pObject) {
    // 获取对象头部的 vptr (指向虚表的指针)

```

```

void* pVirtualTableAddr = *(void**)pObject;
void* FnAddress = nullptr;

// 解析虚表内容
VTFList VirtualTableFuncList = GetClassVirtualTable(pVirtualTableAddr);

std::cout << std::hex;
std::cout << "Instance Object Address: " << pObject << std::endl;
std::cout << "Instance Object Virtual Table Address: " << pVirtualTableAddr << std::endl;
std::cout << "Instance Object Virtual Table Context Overall: " << std::endl;

// 打印每个虚函数的地址
for (int i = 0; i < VirtualTableFuncList.size(); i++)
    std::cout << "Virtual Table Pos:" << std::dec << i + 1 << "    Address: " << std::hex
<< VirtualTableFuncList[i] << std::endl;

std::cout << "Instance Object Pointer Map: " << std::endl;
std::cout << pObject << " -> " << pVirtualTableAddr << " -> {";

// 打印映射关系
for (int i = 0; i < VirtualTableFuncList.size(); i++) {
    std::cout << VirtualTableFuncList[i];
    if (i != VirtualTableFuncList.size() - 1) std::cout << ", ";
}
std::cout << "}" << std::endl;
}

// ===== 劫持配置结构体 =====
struct VirtualFnData {
    int Index;        // 要劫持的虚函数索引 (0=Add, 1=Sub)
    void* FuncAddr; // 新的函数地址 (Hook 函数)
};

/**
 * @brief 核心劫持函数：伪造虚表并替换 vptr
 * @param pObject 目标对象指针
 * @param VirtualFn 劫持配置 (索引 + 新函数)
 * @param ReleaseHeap 是否释放旧的堆内存 (逻辑有缺陷，仅演示用)
 *
 * @process
 * 1. 读取当前虚表内容到 vector
 * 2. 修改 vector 中指定索引的函数地址
 * 3. 在堆上 calloc 一块新内存，复制修改后的虚表
 * 4. 将对象的 vptr 指向这块新内存 (完成劫持)

```

```

*/
bool CheatClassVirtualFunc(void* pObject, VirtualFnData VirtualFn, bool ReleaseHeap) {
    if (!pObject) return false;

    // 1. 获取当前虚表内容 (注意: 这里每次都会重新解析, 可能读到非法内存)
    VTFList VirtualTableFuncList = GetClassVirtualTable(*(void**)pObject);
    int VirtualFnCount = VirtualTableFuncList.size();

    // 检查索引是否越界
    if (VirtualFnCount < VirtualFn.Index) return false;

    // 2. 在内存列表中替换函数地址
    VirtualTableFuncList[VirtualFn.Index] = VirtualFn.FuncAddr;

    // 3. 【关键】在堆上分配可写内存作为"伪造的虚表"
    // 注意: 这里硬编码了 3 个指针大小, 应使用 VirtualFnCount
    void* pVirtualFunctionAddr = calloc(3, sizeof(void*));
    if (!pVirtualFunctionAddr) return false;

    // 复制修改后的虚表内容到堆内存
    memcpy(pVirtualFunctionAddr, VirtualTableFuncList.data(), sizeof(void*) *
VirtualFnCount);

    // 4. 【可选】尝试释放旧内存 (逻辑错误: *(void**)pObject 指向的是 .rdta 只读段或
    上一次的堆, 不能直接 free)
    if (ReleaseHeap)
        free(*(void**)pObject); // 危险: 如果指向 .rdta 会崩溃, 如果指向旧堆且未保存
    也会出错

    // 5. 【偷梁换柱】修改对象头部的 vptr, 指向我们伪造的堆内存
    *(void**)pObject = pVirtualFunctionAddr;

    return true;
}

int main() {
    // 创建对象 (在堆上, 方便控制生命周期)
    DerivedTarget* Dtg = new DerivedTarget();
    VTFList New, Old;

    std::cout << "[ Reverse Engineering Progject : Virtual Table Hooking ]\n" << std::endl;
    std::cout << "=== [Before Hook] ===" << std::endl;

    // 保存原始虚表信息 (用于对比)

```

```

Old = GetClassVirtualTable(*(void**)Dtg);
DisplayClassVirtualFunc(Dtg);

// 测试正常逻辑
Dtg->Add(10.0);
std::cout << "After Add(10): " << std::dec << Dtg->GetValue() << std::hex << std::endl;
// 应为 10

Dtg->Sub(5.0);
std::cout << "After Sub(5): " << std::dec << Dtg->GetValue() << std::hex << std::endl;
// 应为 5

// ===== 开始劫持 =====
VirtualFnData VFD;

// 第一次劫持：将索引 0 (Add) 替换为 Multiply
VFD.Index = 0;
VFD.FuncAddr = &Multiply;
// ReleaseHeap=false: 不尝试释放旧表 (因为旧表在 .rdata, 不能 free)
CheatClassVirtualFunc(Dtg, VFD, false);

// 第二次劫持：将索引 1 (Sub) 替换为 Divide
// 注意：由于 CheatClassVirtualFunc 内部逻辑缺陷, 这次操作可能会重置索引 0 的修
改!
VFD.Index = 1;
VFD.FuncAddr = &Divide;
// ReleaseHeap=true: 尝试释放 (这里会尝试 free 上一次 calloc 的内存, 逻辑勉强成
立但非常脆弱)
CheatClassVirtualFunc(Dtg, VFD, true);

std::cout << "\n=== [After Hook] ===" << std::endl;
New = GetClassVirtualTable(*(void**)Dtg);
DisplayClassVirtualFunc(Dtg);

// 测试劫持后的逻辑
// 预期：Add 被劫持为 Multiply (乘 10), Sub 被劫持为 Divide (除 5)
// 当前值是 5.0
Dtg->Add(10.0);
std::cout << "After Hooked Add(10) [Should be *10]: " << std::dec << Dtg->GetValue()
<< std::hex << std::endl;

Dtg->Sub(5.0);
std::cout << "After Hooked Sub(5) [Should be /5]: " << std::dec << Dtg->GetValue() <<
std::hex << std::endl;

```

```
std::cout << std::endl;

// 打印 加、减、乘、除 函数表
std::cout << "=== [Function Address] ===" << std::endl;
std::cout << std::hex;
std::cout << "Add: " << Old[0] /* Add */ << std::endl;
std::cout << "Sub: " << Old[1] /* Sub */ << std::endl;
std::cout << "Multiply: " << New[0] /* Multiply */ << std::endl;
std::cout << "Divide: " << New[1] /*Divide*/ << std::endl;


// 清理内存 (防止泄漏)
// 实际项目中需要恢复 vptr 并 delete[] 伪造的表
delete Dtg;

return 0;
}
```